Master’s degree in Industrial Electronics Engineering and Computers

Embedded Systems and Computers

GR0040-based SoC

Group 8

André Silva | A104863

Tiago Silva | PG60218

Project supervised by:

Professor Adriano Tavares

Content

1 Analysis 7
1.1 Problem Statement 7
1.2 Requirements 7
1.2.1 Functional requirements 7
1.2.2 Non-functional requirements 7
1.3 Constraints 7
1.3.1 Technical constraints 8
1.3.2 Non-technical constraints 8
1.4 High-level system description 8
1.5 Instruction Set Architecture 8
1.5.1 Instruction formats 8
1.5.2 Instruction Set 10
2 Design 11
2.1 GR0040 11
2.1.1 Inputs/outputs 11
2.1.2 Register File 12
2.1.3 Refactoring 12
2.1.4 Control Unit 13
2.1.5 Datapath 14
2.2 GR0041 15
2.2.1 Inputs/Outputs 15
2.3 Memory organization 16
2.4 Peripheral bus 17
2.5 16-bit Counter/Timer 18
2.5.1 Inputs/Outputs 18
2.5.2 Configuration Register 18
2.5.3 Interrupt Request Register 19
2.6 8-bit Parallel I/O Port 19
2.6.1 Inputs/Outputs 19
2.7 Interrupt Controller 20
2.7.1 Inputs/Outputs 20

2.8 UART 20
2.8.1 Inputs/Outputs 20
2.8.2 Baudrate Generator 21
2.8.3 Receiver 21
2.8.4 Transmitter 22
3 Implementation 23
3.1 GR0040 23
3.1.1 Top-level module 23
3.1.2 Control Unit 24
3.1.3 Datapath 26
3.2 GR0041 28
3.3 Memory 30
3.4 Peripheral Bus 30
3.4.1 Bus encoder 30
3.4.2 Bus decoder 31
3.5 16-bit Counter/Timer 31
3.6 8-bit Parallel I/O Port 33
3.7 Interrupt Controller 33
3.8 UART 36
3.8.1 Baudrate Generator 38
3.8.2 Receiver 39
3.8.3 Transmitter 40
3.9 Top-level module 40
3.10 Python Assembler 43
4 Verification 45
4.1 Test Code 45
4.2 Vivado Block Design 47
4.3 Results 47
5 Conclusion 48
6 References 49

Figures List

Figure 1- Register-register format 9
Figure 2- Register-immediate format 9
Figure 3- Register-register-immediate format 9
Figure 4- 12-bit immediate (i12) format 9
Figure 5- Branch (br) format 9
Figure 6- Instruction Set 10
Figure 7- GR0040 inputs/outputs 11
Figure 8- Processor refactoring 13
Figure 9- Datapath 14
Figure 10- GR0041 Inputs/outputs 15
Figure 11- High-level memory map 16
Figure 12- Memory aliasing example 16
Figure 13- Memory map 17
Figure 14- Peripheral bus encoder/decoder inputs/outputs 18
Figure 15- 16-bit Timer/Counter inputs/outputs 18
Figure 16- 16-bit Timer/Counter configuration register 19
Figure 17- 16-bit Timer/Counter interrupt request register 19
Figure 18- 8-bit Parallel I/O Port inputs/outputs 19
Figure 19- Interrupt Controller inputs/outputs 20
Figure 20- UART inputs/outputs 20
Figure 21- UART sub-modules 21
Figure 22- UART Receiver FSM 22
Figure 23- UART Transmitter FSM 22
Figure 24- gr0040.v (1) 23
Figure 25- gr0040.v (2) 24
Figure 26- controlUnit.v (1) 25
Figure 27- controlUnit.v (2) 25
Figure 28- Register File IP Customization 26
Figure 29- datapath.v (1) 26
Figure 30- datapath.v (2) 27
Figure 31- datapath.v (3) 28
Figure 32- gr0041.v (1) 29
Figure 33- gr0041.v (2) 29
Figure 34- Memory IP configuration 30
Figure 35- ctrl_enc.v 30
Figure 36- ctrl_dec.v 31
Figure 37- timer.v (1) 31
Figure 38- timer.v (2) 32
Figure 39- pario.v 33
Figure 40- pic.v (1) 34

Figure 41- pic.v (2) 34
Figure 42- pic.v (3) 35
Figure 43- UartController.v (1) 36
Figure 44- UartController.v (2) 37
Figure 45- UartController.v (3) 37
Figure 46- BaudRateGen.v..... 38
Figure 47- UartReceiver.v 39
Figure 48- UartTransmitter.v 40
Figure 49- soc.v (1) 41
Figure 50- soc.v (2) 42
Figure 51- soc.v (3) 42
Figure 52- assembler.py (1) 43
Figure 53- assembler.py (2) 43
Figure 54- assembler.py (3) 44
Figure 55- assembler.py (4) 44
Figure 56- Assembly test code (1) 45
Figure 57- Assembly test code (2) 46
Figure 58- Assembly test code (3) 46
Figure 59- Block Design (Vivado)..... 47
Figure 60- Echo test using GTKTerm..... 47
Figure 61- LED toggling every 500 timer interrupts 47
Figure 62- LED toggling when push-button is pressed 47

Acronyms

SOC	System-on-Chip
FPGA	Field-Programmable Gate Array
GPIO	General-Purpose Input/Output
ALU	Arithmetic Logic Unit
UART	Universal Asynchronous Receiver-Transmitter
ISA	Instruction Set Architecture
ISR	Interrupt Service Routine
PC	Program Counter
CPU	Central Processing Unit
MSB	Most Significant Bit
LSB	Least Significant Bit
LED	Light Emitting Diode
FSM	Finite State Machine
RAM	Random Access Memory
KB	Kilobyte

1 Analysis

1.1 Problem Statement

The primary objective of this project is the architectural design, implementation, and verification of a System-on-Chip tailored for FPGA synthesis.

The system is based on the GR0040 CPU, a 16-bit RISC processor core originally designed by Jan Gray. A portion of this work consisted of analyzing and refactoring the original design to meet modern implementation standards.

Furthermore, the core challenge involved architecturally extending the processor's capabilities to support asynchronous event handling. This required the development of a dedicated Interrupt Controller to manage and prioritize multiple requests originating from distinct memory-mapped peripherals, such as the Timer, GPIO, and UART.

1.2 Requirements

To ensure the correctness of the system, a group of functional and non-functional requirements to be met were defined.

1.2.1 Functional requirements

- Support the complete ISA
- Support a load-store architecture
- Support a byte-addressable memory access
- Implement an interrupt controller
- Implement at least one peripheral

1.2.2 Non-functional requirements

- Support extending functionalities

1.3 Constraints

The way the requirements are fulfilled is defined by the project constraints, whether technical or non-technical.

1.3.1 Technical constraints

- Use Zybo Z7-10 board
- Implement in Verilog HDL
- Be compliant with Control Unit/Datapath model

1.3.2 Non-technical constraints

- Groups of two students
- Deadline until the end of the semester

1.4 High-level system description

The system is centered on the GR0040, a simple 16-bit RISC processor core that follows a non-pipelined, two-operand load-store architecture.

To support asynchronous event handling, the core is embedded within the GR0041 system-wrapper. Although it provides the necessary internal logic to suspend execution and accept non-nested interrupt vectors, the actual management and prioritization of these events are offloaded to a dedicated Interrupt Controller.

The System-on-Chip architecture is completed by integrating memory for instruction and data storage, alongside a suite of memory-mapped peripherals: the Interrupt Controller, Timer, GPIO, and UART. These components are interconnected via a unified parallel bus interface, which arbitrates data exchange between the central processor and the peripheral devices.

1.5 Instruction Set Architecture

To support the complete instruction set, it is crucial to have a deep knowledge of its architecture.

1.5.1 Instruction formats

Below are listed in the instruction formats available in this architecture. In every format, the four MSB are reserved for the opcode, with the remaining fields varying to support register-based, immediate, or memory access operations.

[15:12]	[11:8]	[7:4]	[3:0]
op	rd	rs	fn

Figure 1- Register-register format

[15:12]	[11:8]	[7:4]	[3:0]
op	rd	fn	imm

Figure 2- Register-immediate format

[15:12]	[11:8]	[7:4]	[3:0]
op	rd	rs	imm

Figure 3- Register-register-immediate format

[15:12]	[11:0]
op	imm12

Figure 4- 12-bit immediate (i12) format

[15:12]	[11:8]	[7:0]
op	cond	disp8

Figure 5- Branch (br) format

1.5.2 Instruction Set

The instruction set is partitioned into arithmetic and logic operations, memory access handled by Load and Store commands, and control flow managed by branch and jump instructions.

To signal the completion of an ISR to the Interrupt Controller, the original set was extended with the specialized RETI (Return from Interrupt) instruction. It was designed to perform a dual function: to atomically restore the Program Counter from register r0, and to clear the internal "busy" flag, thereby unblocking the logic to accept new interrupt requests.

Below are listed all the supported instructions in this architecture.

Opcode	Format	Assembler	Semantics
0	rri	jal rd, imm (rs)	rd = pc, pc = imm + rs;
1	rri	addi rd,rs,imm	rd = imm + rs;
2	rr	{add sub and xor adc sbc cmp srl sra} rd,rs	[rd=] rd fn rs;
3	ri	{- rsubi adci rsbci andi xori rscmpi} rd,imm	[rd=] imm fn rd;
4	rri	lw rd,imm(rs)	rd = *(int*)(imm+rs);
5	rri	lb rd,imm(rs)	rd = *(byte*)(imm+rs);
6	rri	sw rd,imm(rs)	*(int*)(imm+rs) = rd;
7	rri	sb rd,imm(rs)	*(byte*)(imm+rs) = rd;
8	i12	imm imm12	imm'next[15:4] = imm12;
9	br	{br brn beq bne bc bnc bv bnv blt bge ble bgt bltu bgeu bleu bgtu} label	if(cond) pc += 2*disp8
A	-	reti r0	pc = r0

Figure 6- Instruction Set

2 Design

2.1 GR0040

2.1.1 Inputs/outputs

The processor interfaces with the rest of the system through a set of signals grouped into system controls, code memory access, and data memory access, as illustrated in Figure 1.

- **clk (input) -** clock signal that synchronizes all operations
- **rst (input) -** resets the PC and internal registers to a known state
- **i_ad (output) -** points to the next instruction to be fetched from memory
- **insn (input) -** receives the 16-bit instruction specified by *i_ad*
- **d_ad (output) -** data address bus
- **data (inout) -** bidirectional data bus to transfer information between CPU's internal registers and external devices
- **do (output) -** carries the value from a source register to be written to memory during a store operation
- **lw/lb (outputs) -** indicate a read operation from memory
- **sw/sb (outputs) -** indicate a write operation to memory
- **insn_ce (output) -** used to stall the fetch stage when necessary
- **int_req (input) -** interrupt request
- **vector_in (input) -** the address of the ISR provided by the Interrupt Controller

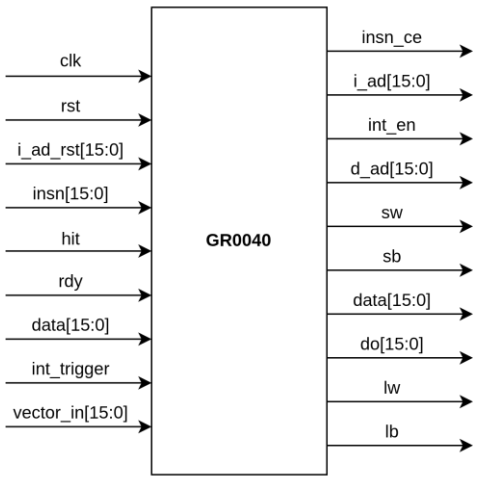

Figure 7- GR0040 inputs/outputs

2.1.2 Register File

The 16x16-bit Register File serves as the central storage unit of the processor, embedded within the Datapath to ensure immediate operand availability for the ALU.

The architectural reliance on a single bank of dual-port memory fundamentally dictates the processor's two-operand execution model, as the structure allows for only two distinct access mechanisms: a primary read-write port addressed by the destination register field *(rd)* and a secondary read-only port addressed by the source register field *(rs)*.

While the registers are architecturally general-purpose, the design adheres to software conventions, most notably the reservation of *r0* to store the return address during interrupt service routines.

2.1.3 Refactoring

A crucial component of the project work involved the architectural refactoring of the original processor, transforming a highly optimized but unstructured design into a modular system that is compliant with the Control Unit and Datapath separation model.

This reorganization segregates the design responsibilities, beginning with an instruction slicing stage that decomposes the 16-bit instruction word into different instruction fields. The Control Unit functions as the processor's command center and dictates which operation the Datapath must execute.

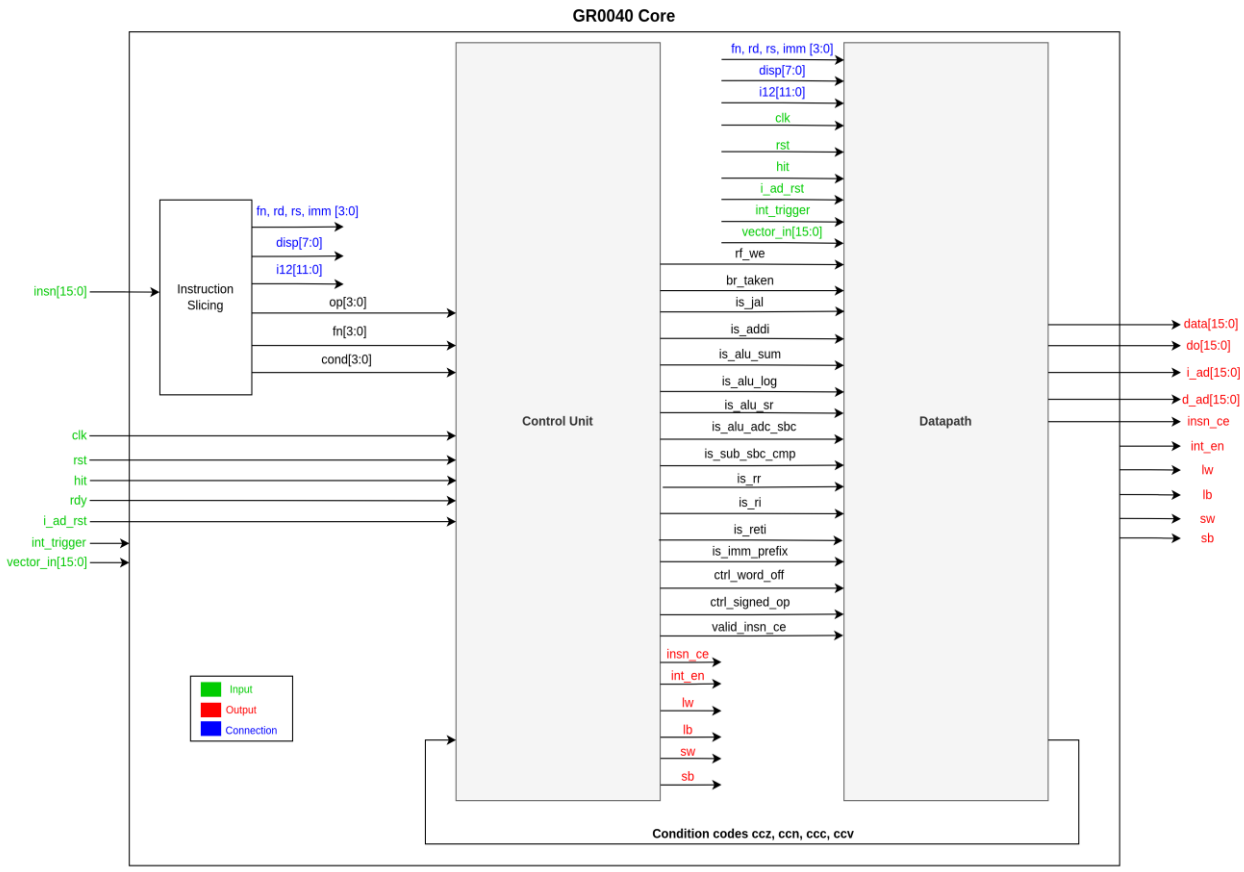

Figure 8- Processor refactoring

2.1.4 Control Unit

The Control Unit receives as an input the *op*, *fn*, and *cond* fields from the instruction slicing stage, alongside critical system status flags such as *hit* and *rdy* which indicate instruction validity and memory availability, and the condition codes.

Based on this, the Control Unit generates control signals directed at the Datapath, effectively configuring the execution path for each cycle. These outputs include the Register File write-enable signal *(rf_we)*, branch decision flags *(br_taken)*, and specific operation selectors that drive the internal multiplexers.

Furthermore, the Control Unit manages the external system interface by asserting the memory access signals *lw*, *lb*, *sw*, and *sb*, while simultaneously handling the interrupt logic through the *int_trigger* input and the *int_en* output to coordinate with the Interrupt Controller.

2.1.5 Datapath

The Datapath encapsulates the physical processing resources of the processor, executing the operations dictated by the Control Unit through logic gates, arithmetic modules, and multiplexers.

The arithmetic and logical core of the Datapath is built around a specialized *addsub* module responsible for mathematical operations, alongside parallel logic blocks dedicated to bitwise functions and shift operations. The results from these units feed into the condition code registers *(ccz, ccn, ccc, ccv)*, which capture the processor's status flags *(Zero, Negative, Carry, and Overflow)* to inform future branching decisions. The final stage of the Datapath consists of a cascaded structure of multiplexers that select the appropriate result to drive the data bus, whether it be an ALU calculation, a return address, or a memory value.

Additionally, the design integrates interrupt handling logic directly into the PC path, utilizing the *vector_in* signal to force the program counter to a specific memory address when an external *int_trigger* event is acknowledged.

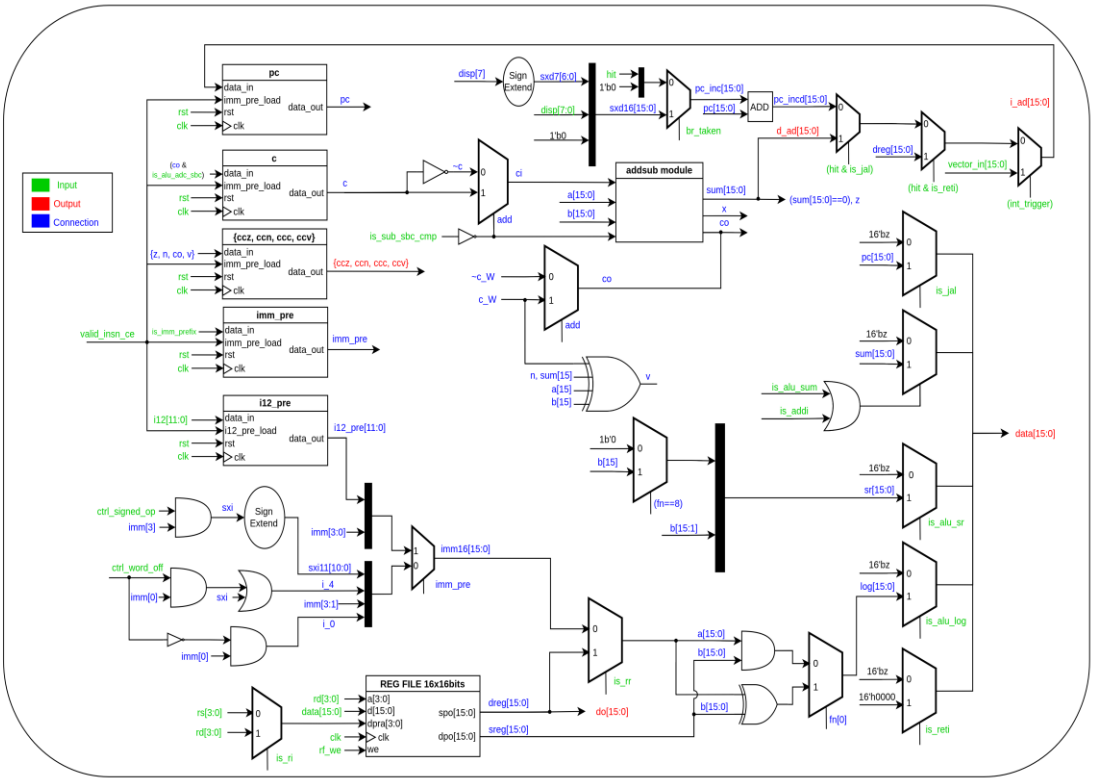

Figure 9- Datapath

2.2 GR0041

The GR0041 serves as an architectural wrapper that encapsulates the GR0040 processor core. Its primary responsibility is to bridge the functionality gap by implementing the high-level interrupt management logic absent in the base core.

Functioning as a state-aware gatekeeper, this module receives external requests from the Interrupt Controller and arbitrates their execution based on the processor's status. The design mandates a non-nested interrupt policy, ensuring that ISRs are executed atomically without interruption. Upon accepting an incoming request, the wrapper transitions to a "busy" state, effectively masking any subsequent interrupt signals to prevent reentrancy. This lock is maintained until the system detects the execution of the RETI instruction on the instruction bus, which signals the completion of the ISR. Additionally, the interface is designed to accept an external interrupt vector *(vector_in)*, which is injected into the GR0040 to redirect the Program Counter to the appropriate ISR address.

2.2.1 Inputs/Outputs

The GR0041 receives an interrupt request *(int_req)* and decides whether the GR0040 must be interrupted. All the other signals are transparently propagated directly to and from the GR0040 core.

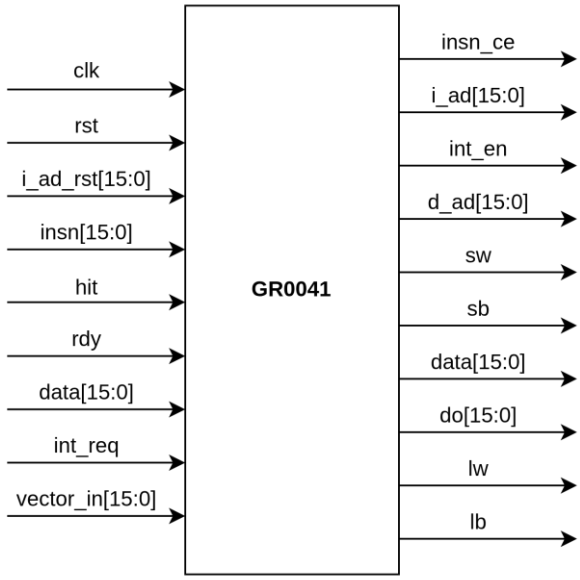

Figure 10- GR0041 Inputs/outputs

2.3 Memory organization

The system utilizes a 16-bit address space, providing a total addressable range of 64KB. As illustrated in the high-level memory organization, this space is logically partitioned into two distinct 32KB regions based on the state of the MSB of the address bus. The lower half is dedicated to code and data storage, while the upper half is reserved for the peripherals.

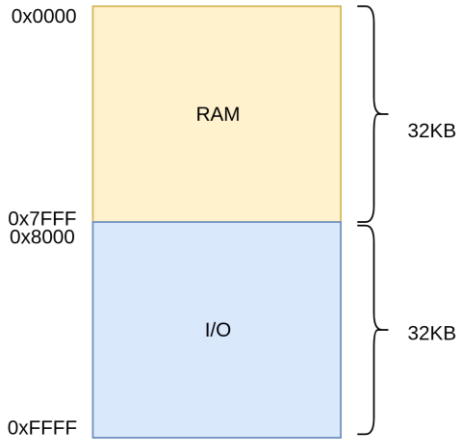

Figure 11- High-level memory map

Although the architecture supports up to 32KB of code and data memory, it was decided to instantiate only 1KB of dual-port Block RAM. A consequence of this decision is the phenomenon of memory aliasing. As depicted in the figure below, partial decoding *(d_ad[9:1])* causes the 1KB memory block to be mirrored repeatedly throughout the lower 32KB range. For example, writing data to address *0x0000* is physically identical to writing to *0x0400*, as they map to the same physical storage cells.

d_ad	[15]	[14:10]	[9:1]	[0]
(0x0000)	0	00000	000000000	0
(0x0400)	0	00001	000000000	0

Figure 12- Memory aliasing example

Knowing this, a detailed memory map was designed. The interrupt vector table is located at the very bottom, containing the hardcoded addresses for the ISRs. The

processor's reset entry point is configured at *0x0020*, marking the beginning of the user application code.

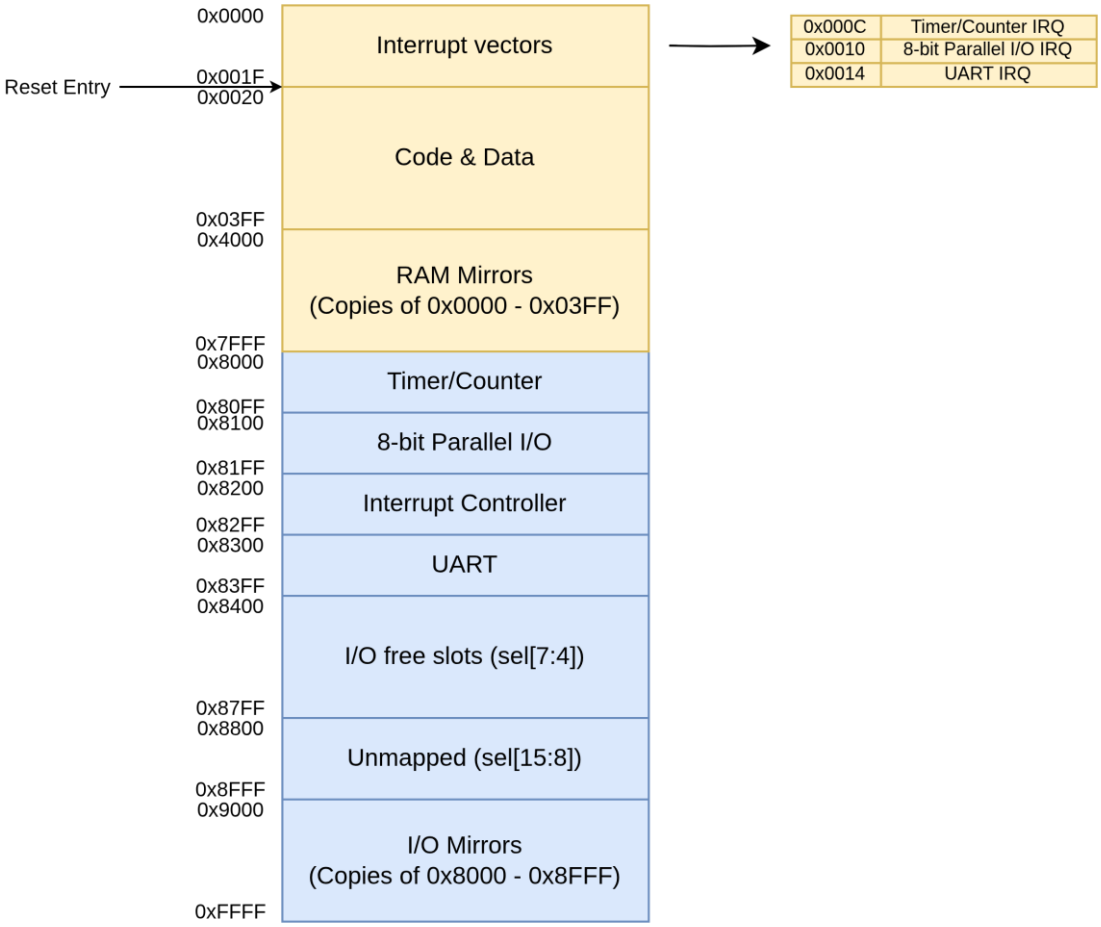

Figure 13- Memory map

2.4 Peripheral bus

The system's peripheral bus relies on a paired encoder/decoder mechanism to manage system communication.

The bus encoder functions as a protocol aggregator. It monitors the processor's memory interface signals and the I/O address flags and compresses this information into an abstract control vector *ctrl* while simultaneously decoding the address target into a select vector *sel*.

On the device side, each peripheral instantiates a bus decoder. This module receives the abstract *ctrl* and *sel* vectors and reconstructs the signals required for operation.

By abstracting the control interface, this design ensures that enhancements to the bus protocol remain transparent to the peripheral cores, thereby preserving their reusability.

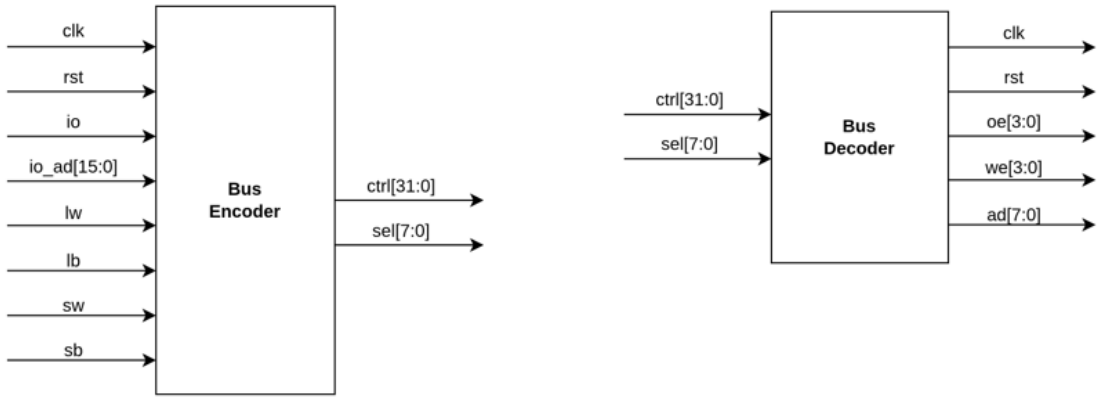

Figure 14- Peripheral bus encoder/decoder inputs/outputs

2.5 16-bit Counter/Timer

2.5.1 Inputs/Outputs

Beyond the standard bus decoder interface signals the additional signals include the *i* input, which functions as the external event trigger or clock enable depending on the operating mode, the 16-bit *cnt_init* input used to automatically reload the counter upon overflow, and the *irq* output, which is asserted to signal an interrupt request.

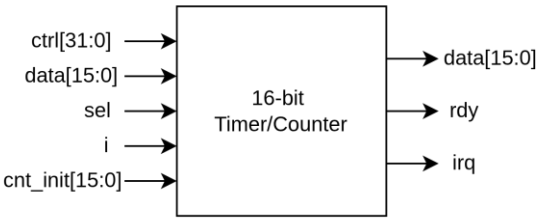

Figure 15- 16-bit Timer/Counter inputs/outputs

2.5.2 Configuration Register

The module's fundamental behavior is configured via configuration register *(CR0)*, located at memory address *0x8000*. The bit 0 is the operation mode selector,

while bit 1 enables the hardware to generate an interrupt request upon overflow when set.

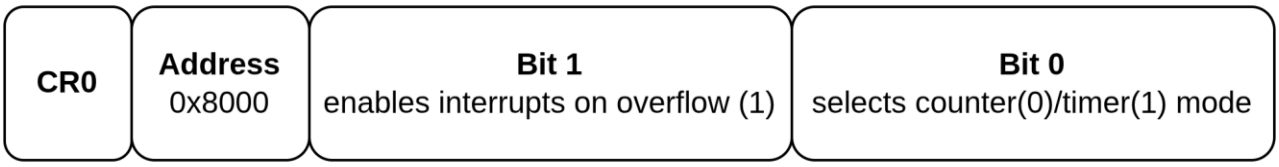

Figure 16- 16-bit Timer/Counter configuration register

2.5.3 Interrupt Request Register

The interrupt request register (CR1) is located at *0x8002*. This register holds the interrupt status flag in bit 0, set to 1 whenever the 16-bit counter overflows.

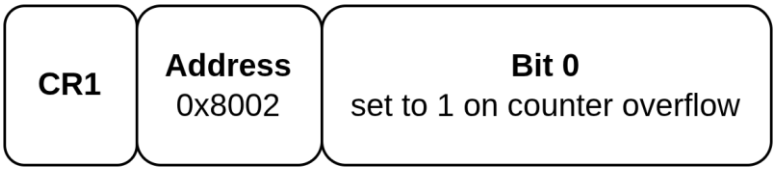

Figure 17- 16-bit Timer/Counter interrupt request register

2.6 8-bit Parallel I/O Port

2.6.1 Inputs/Outputs

The Parallel I/O module operates as a GPIO interface. During read operations, such as monitoring the status of a push button, the module samples the external input signals and drives them directly onto the system data bus. Conversely, during write operations, such as controlling an LED, the module latches the data provided by the processor to continuously drive the external output pins.

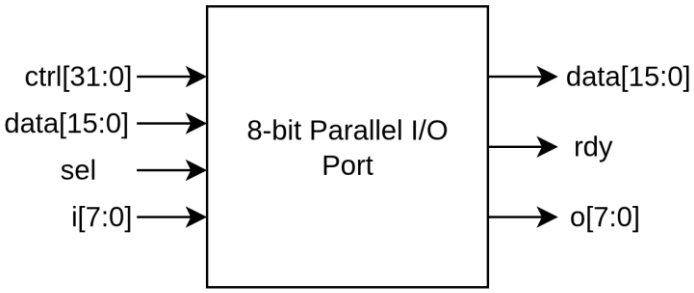

Figure 18- 8-bit Parallel I/O Port inputs/outputs

2.7 Interrupt Controller

2.7.1 Inputs/Outputs

The Interrupt Controller primary function is to aggregate asynchronous interrupt requests from multiple hardware sources (Timer, Parallel I/O, UART) and strictly order their execution based on a programmable priority scheme.

As illustrated in the figure below, the module receives the interrupt vector *irq_in[7:0]*, where each bit corresponds to a hardware source. On the processor side, it drives two critical signals: the *int_req* line, which signals the presence of a valid interrupt, and the *vector_out[15:0]*, which provides the memory address of the target ISR.

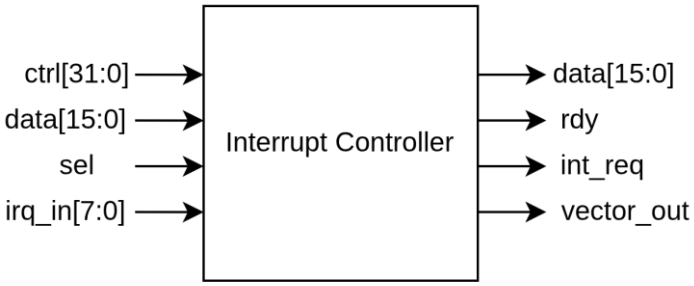

Figure 19- Interrupt Controller inputs/outputs

2.8 UART

2.8.1 Inputs/Outputs

The UART peripheral was designed to enable full-duplex serial communication, featuring specific logic to generate a system interrupt request immediately upon the successful reception of a data byte.

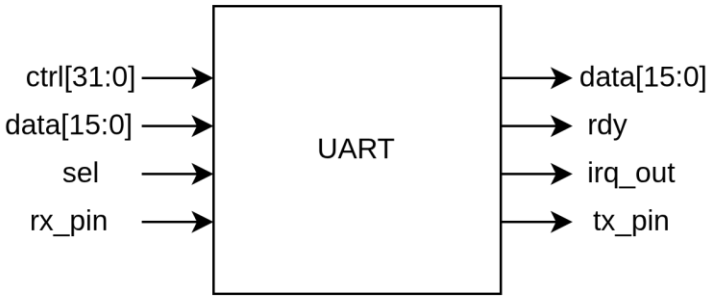

Figure 20- UART inputs/outputs

Independent transmission and reception are allowed by decoupling the logic into three synchronized sub-modules: the Baudrate Generator, the Receiver, and the Transmitter.

2.8.2 Baudrate Generator

The Baudrate Generator functions as the timing core, accepting a 16-bit divisor to down-convert the system clock into a precise *baud_tick* pulse, which synchronizes both the Transmitter and Receiver logic.

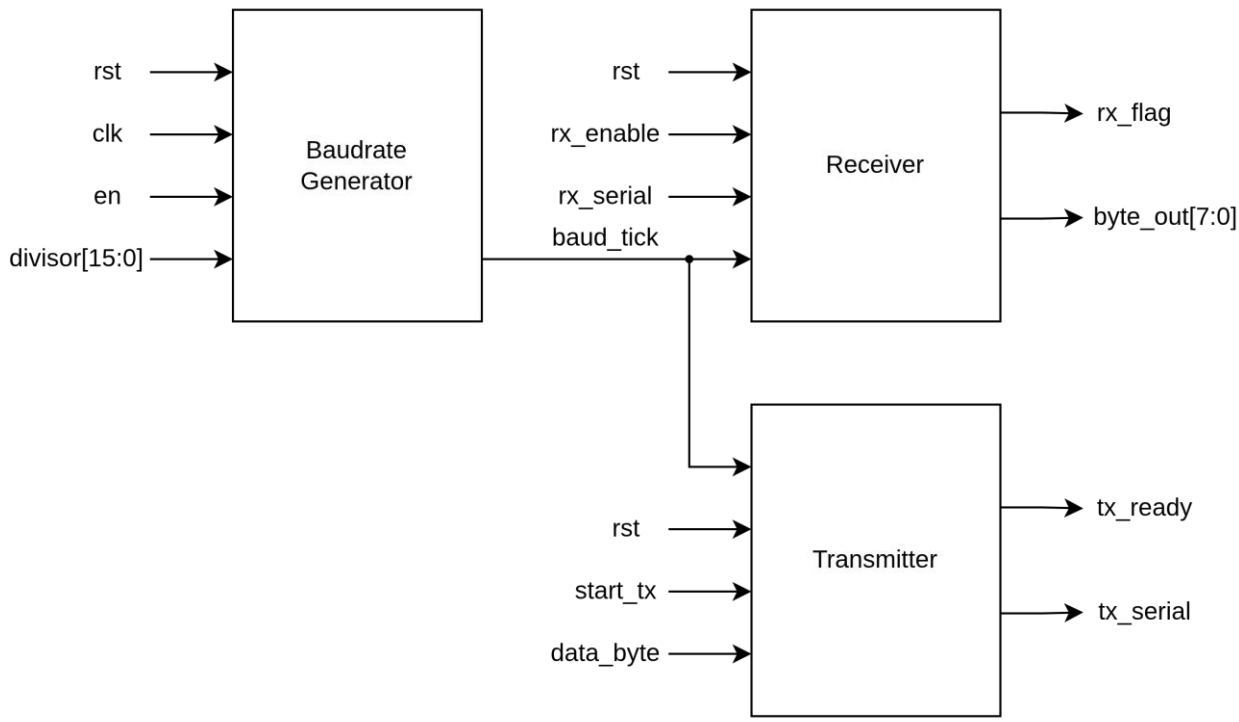

Figure 21- UART sub-modules

2.8.3 Receiver

The Receiver's FSM begins in *STATE_WAIT*, monitoring the *rx_serial* line for a falling edge (start bit). Once detected, it moves to *STATE_READ*, where it reads the incoming bits. After capturing 8 bits, it transitions to STATE_END to assert the *rx_flag*, indicating that valid data is available for the processor to read.

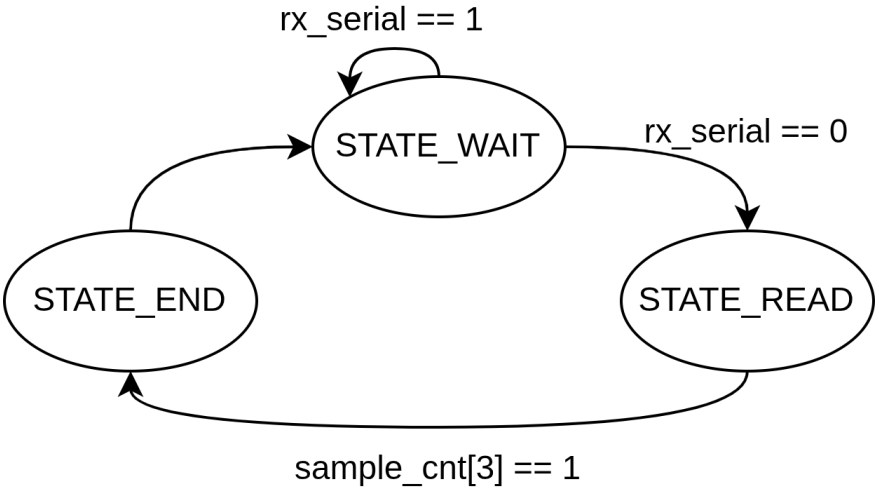

Figure 22- UART Receiver FSM

2.8.4 Transmitter

The Transmitter's FSM remains in *STATE_IDLE* until data is written to the buffer. Upon the *start_tx* signal, it transitions to *STATE_START* to drive the line low, proceeds to *STATE_SHIFT* to serialize the 8 data bits, and concludes in STATE_STOP by driving the line high to assert the stop bit.

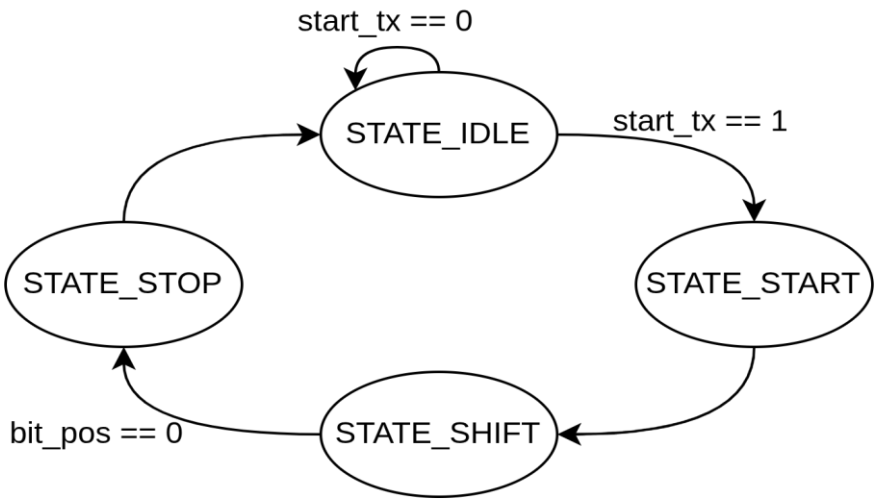

Figure 23- UART Transmitter FSM

3 Implementation

3.1 GR0040

3.1.1 Top-level module

The top-level module is responsible for instantiating the Control Unit and Datapath, bridging the control and state signals. Its primary logical function is Instruction Slicing, where the 16-bit instruction word is decomposed into fields.

```
`include "defines.vh"

module gr0040(
    input  clk, rst,
    input [`AN:0] i_ad_rst,
    input  [`IN:0] insn,
    input  hit,
    input  rdy,
    inout  [`N:0] data,
    input int_trigger,
    input [15:0] vector_in,
    output insn_ce,
    output [`AN:0] i_ad,
    output int_en,
    output [`AN:0] d_ad,
    output sw, sb,
    output [`N:0] do,
    output lw, lb
);
    //instruction slicing
    wire [3:0] op   = insn[15:12];
    wire [3:0] rd   = insn[11:8];
    wire [3:0] rs   = insn[7:4];
    wire [3:0] imm  = insn[3:0];
    wire [3:0] cond = insn[11:8];
    wire [11:0] i12 = insn[11:0];
    wire [7:0] disp = insn[7:0];
    wire [3:0] fn   = `RI ? insn[7:4] : insn[3:0];

    //control signals
    wire rf_we, br_taken;
    wire ccz, ccn, ccc, ccv;
    wire is_jal, is_addi, is_rr, is_ri, is_imm_prefix;
    wire is_alu_sum, is_alu_log, is_alu_sr;
    wire is_alu_adc_sbc, is_sub_sbc_cmp;
    wire is_reti;
    wire valid_insn_ce;
    wire ctrl_word_off;
    wire ctrl_signed_op;
```

Figure 24- gr0040.v (1)

```
41      ControlUnit cu (
42          .clk(clk), .rst(rst),
43          .op(op), .fn(fn), .cond(cond),
44          .hit(hit), .rdy(rdy),
45          .ccz(ccz), .ccn(ccn), .ccc(ccc), .ccv(ccv),
46          .valid_insn_ce(valid_insn_ce),
47          .ctrl_word_off(ctrl_word_off),
48          .ctrl_signed_op(ctrl_signed_op),
49          .is_rr(is_rr),
50          .is_ri(is_ri),
51          .is_imm_prefix(is_imm_prefix),
52          .rf_we(rf_we), .br_taken(br_taken),
53          .is_jal(is_jal), .is_addi(is_addi),
54          .is_alu_sum(is_alu_sum), .is_alu_log(is_alu_log), .is_alu_sr(is_alu_sr),
55          .is_alu_adc_sbc(is_alu_adc_sbc), .is_sub_sbc_cmp(is_sub_sbc_cmp),
56          .is_reti(is_reti),
57          .insn_ce(insn_ce), .int_en(int_en),
58          .lw(lw), .lb(lb), .sw(sw), .sb(sb)
59      );
60
61      Datapath dp (
62          .clk(clk), .rst(rst),
63          .fn(fn),
64          .rd(rd), .rs(rs),
65          .imm(imm), .i12(i12), .disp(disp),
66          .hit(hit), .i_ad_rst(i_ad_rst),
67          .valid_insn_ce(valid_insn_ce),
68          .ctrl_word_off(ctrl_word_off),
69          .ctrl_signed_op(ctrl_signed_op),
70          .is_rr(is_rr),
71          .is_ri(is_ri),
72          .is_imm_prefix(is_imm_prefix),
73          .rf_we(rf_we), .br_taken(br_taken),
74          .is_jal(is_jal), .is_addi(is_addi),
75          .is_alu_sum(is_alu_sum), .is_alu_log(is_alu_log), .is_alu_sr(is_alu_sr),
76          .is_alu_adc_sbc(is_alu_adc_sbc), .is_sub_sbc_cmp(is_sub_sbc_cmp),
77          .is_reti(is_reti),
78          .int_trigger(int_trigger),.vector_in(vector_in),
79          .ccz(ccz), .ccn(ccn), .ccc(ccc), .ccv(ccv),
80          .i_ad(i_ad), .d_ad(d_ad), .do(do), .data(data)
81      );
82  endmodule
```

Figure 25- gr0040.v (2)

3.1.2 Control Unit

In the control unit, instruction decoding generates critical signals such as *rf_we* (register file write enable) and ALU selectors. Branch decision logic is implemented using a case statement that compares the status flags *(ccz, ccn, ccc, ccv)* against the instruction's *cond* field.

```
1   `include "defines.vh"
2
3   module ControlUnit(
4       input  clk, rst,
5       input [3:0] op, fn, cond,
6       input  hit, rdy,
7       input  ccz, ccn, ccc, ccv,
8       output valid_insn_ce,
9       output rf_we,
10      output br_taken,
11      output is_jal, is_addi,
12      output is_alu_sum, is_alu_log,
13      output is_alu_sr,
14      output is_alu_adc_sbc,
15      output is_sub_sbc_cmp,
16      output is_reti,
17
18      output is_rr,
19      output is_ri,
20      output is_imm_prefix,
21      output ctrl_word_off,
22      output ctrl_signed_op,
23
24      output insn_ce,
25      output int_en,
26      output lw, lb, sw, sb
27  );
28
29      wire mem = hit & (`LB|`LW|`SB|`SW);
30      assign insn_ce = rst | ~(mem & ~rdy);
31      assign valid_insn_ce = hit & insn_ce;
32
33      //register file write enable
34      assign rf_we = valid_insn_ce & ~rst & ((`ALU & ~`CMP)|`ADDI|`LB|`LW|`JAL|`RETI);
35
36      //decoding signals
37      assign is_rr          = `RR;
38      assign is_ri          = `RI;
39      assign is_imm_prefix  = `IMM;
40      assign ctrl_word_off  = `LW | `SW | `JAL;
41      assign ctrl_signed_op = `ADDI | `ALU;
42
```

Figure 26- controlUnit.v (1)

```
43      //ALU controls
44      assign is_jal         = `JAL;
45      assign is_addi        = `ADDI;
46      assign is_alu_sum     = `ALU & `SUM;
47      assign is_alu_log     = `ALU & `LOG;
48      assign is_alu_sr      = `ALU & `SR;
49      assign is_alu_adc_sbc = `ALU & (`ADC | `SBC);
50      assign is_sub_sbc_cmp = `ALU & (`SUB | `SBC | `CMP);
51      assign is_reti = `RETI;
52
53      //memory outputs
54      assign lw = hit & `LW;
55      assign lb = hit & `LB;
56      assign sw = hit & `SW;
57      assign sb = hit & `SB;
58      assign int_en = hit & ~(`IMM | `ALU & (`ADC | `SBC | `CMP));
59
60      //branch logic
61      reg t;
62      always @(*) begin
63          case (cond & 4'b1110)
64              `BR:   t = 1;
65              `BEQ:  t = ccz;
66              `BC:   t = ccc;
67              `BV:   t = ccv;
68              `BLT:  t = ccn ^ ccv;
69              `BLE:  t = (ccn ^ ccv) | ccz;
70              `BLTU: t = ~ccz & ~ccc;
71              `BLEU: t = ccz | ~ccc;
72              default: t = 0;
73          endcase
74      end
75      assign br_taken = hit & `Bx & (cond[0] ? ~t : t);
76
77  endmodule
```

Figure 27- controlUnit.v (2)

3.1.3 Datapath

To implement the register file, it was used a Distributed Memory Generator block from Vivado's IP Catalog to be a Dual Port RAM with a depth of 16 registers, each one 16 bits wide.

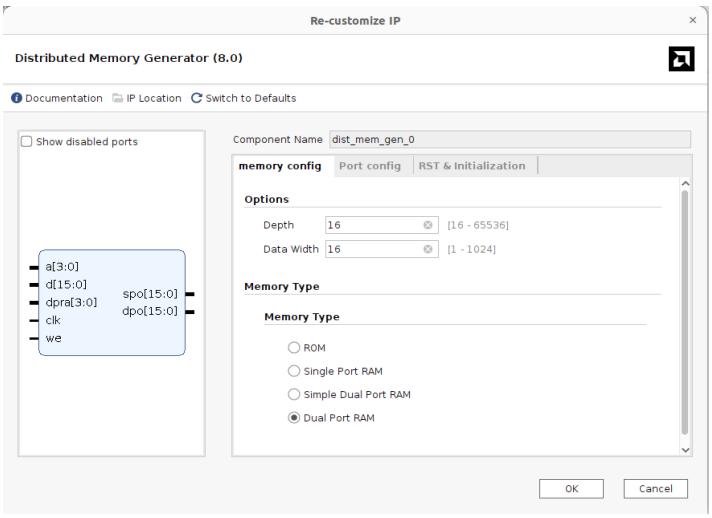

Figure 28- Register File IP Customization

The register file is instantiated in the Datapath module.

```
`include "defines.vh"

module Datapath(
    input  clk, rst,
    input [3:0] fn, rd, rs, imm,
    input [11:0] i12,
    input [7:0] disp,
    input  hit,
    input  [`AN:0] i_ad_rst,
    input  valid_insn_ce,
    input  rf_we, br_taken,
    input  is_jal, is_addi,
    input  is_rr, is_ri,
    input  is_imm_prefix,
    input  ctrl_word_off,
    input  ctrl_signed_op,
    input  is_alu_sum, is_alu_log, is_alu_sr,
    input  is_alu_adc_sbc, is_sub_sbc_cmp,
    input int_trigger,
    input [15:0] vector_in,
    input is_reti,

    output reg ccz, ccn, ccc, ccv,
    output [`AN:0] i_ad,
    output [`AN:0] d_ad,
    output [`N:0] do,
    inout  [`N:0] data
);

    wire [`N:0] dreg, sreg;
    wire [3:0] reg_addr_b = is_ri ? rd : rs;

    dist_mem_gen_0 regfile (
        .a(rd),
        .d(data),
        .dpra(reg_addr_b),
        .clk(clk),
        .we(rf_we),
        .spo(dreg),
        .dpo(sreg)
    );
    assign do = dreg;
```

Figure 29- datapath.v (1)

The following segment details the logic for immediate operand reconstruction, the instantiation of the optimized *addsub* module, and the subsequent derivation of condition code flags used for branching decisions.

```
44      //immediate generation
45      reg imm_pre;
46      reg [11:0] i12_pre;
47
48      always @(posedge clk) begin
49          if (rst) imm_pre <= 0;
50          else if (valid_insn_ce) imm_pre <= is_imm_prefix;
51      end
52      always @(posedge clk) begin
53          if (valid_insn_ce) i12_pre <= i12;
54      end
55
56      wire sxi = ctrl_signed_op & imm[3];
57      wire [10:0] sxi11 = {11{sxi}};
58      wire i_4 = sxi | (ctrl_word_off & imm[0]);
59      wire i_0 = ~ctrl_word_off & imm[0];
60
61      wire [`N:0] imm16 = imm_pre ? {i12_pre, imm} : {sxi11, i_4, imm[3:1], i_0};
62
63      //ALU operations
64      wire [`N:0] a = is_rr ? dreg : imm16;
65      wire [`N:0] b = sreg;
66      wire [`N:0] sum;
67      wire add = ~is_sub_sbc_cmp;
68
69      reg c;
70      wire ci = add ? c : ~c;
71      wire c_W, x;
72
73      addsub adder(.add(add), .ci(ci), .a(a), .b(b), .sum(sum), .x(x), .co(c_W));
74
75      //condition code flags
76      wire z = sum == 0;
77      wire n = sum[`N];
78      wire co = add ? c_W : ~c_W;
79      wire v = c_W ^ sum[`N] ^ a[`N] ^ b[`N];
80
81      always @(posedge clk) begin
82          if (rst) {ccz, ccn, ccc, ccv} <= 0;
83          else if (valid_insn_ce) {ccz, ccn, ccc, ccv} <= {z, n, co, v};
84      end
```

Figure 30- datapath.v (2)

The code below details the result multiplexer that drives the selected result or return address onto the shared bus, and the next-address logic, which resolves the PC's update by prioritizing interrupt vectors, subroutine returns, and branch targets over sequential execution.

```
    always @(posedge clk) begin
        if (rst) c <= 0;
        else if (valid_insn_ce) c <= co & is_alu_adc_sbc;
    end

    //ALU result mux
    wire [`N:0] log = fn[0] ? (a^b) : (a&b);
    wire [`N:0] sr  = {(`SRA ? b[`N] : 0), b[`N:1]};
    wire sum_en = is_alu_sum | is_addi;
    reg [`AN:0] pc;

    assign data = sum_en     ? sum : 16'bz;
    assign data = is_alu_log ? log : 16'bz;
    assign data = is_alu_sr  ? sr  : 16'bz;
    assign data = is_jal     ? pc  : 16'bz;
    assign data = is_reti    ? 16'h0000 : 16'bz;

    wire [6:0] sxd7   = {7{disp[7]}};
    wire [`N:0] sxd16 = {sxd7, disp, 1'b0};
    wire [`N:0] pcinc = br_taken ? sxd16 : {hit, 1'b0};
    wire [`N:0] pcincd = pc + pcinc;


    assign i_ad = (int_trigger)    ? vector_in :  //jump to interrupt vector
                  (hit & is_reti)  ? dreg      :
                  (hit & is_jal)   ? sum       :
                  pcincd;


    assign d_ad = sum;

    always @(posedge clk) begin
        if (rst) pc <= i_ad_rst;
        else if (valid_insn_ce) pc <= i_ad;
    end

endmodule

module addsub(input add, ci, input [15:0] a, b, output [15:0] sum, output x, co);
    assign {co,sum,x} = add ? {a,ci}+{b,1'b1} : {a,ci}-{b,1'b1};
endmodule
```

Figure 31- datapath.v (3)

3.2 GR0041

The GR0041 primary implementation focus is the enforcement of a non-nested interrupt model, which guarantees the atomicity of ISRs.

This module utilizes the flag *isr_active,* asserted synchronously immediately after an internal interrupt trigger (*int*) is accepted, effectively locking the system against further requests. Upon detecting the execution of a RETI, the module clears i*sr_active*, thereby re-arming the system to accept pending events.

```
1   `include "defines.vh"
2
3   module gr0041(
4       input  clk, rst,
5       input [`AN:0] i_ad_rst,
6       input int_req,
7       input [15:0] vector_in,
8       output insn_ce,
9       output [`AN:0] i_ad,
10      input  [`IN:0] insn,
11      input  hit,
12      output zero_insn,
13      output [`AN:0] d_ad,
14      input  rdy,
15      output sw, sb,
16      output [`N:0] do,
17      output lw, lb,
18      inout  [`N:0] data
19  );
20
21      wire int_en;
22      reg int;
23
24      gr0040 core(
25          .clk(clk), .rst(rst),
26          .i_ad_rst(i_ad_rst),
27          .insn(insn),
28          .int_trigger(int),
29          .vector_in(vector_in),
30          .hit(hit | int),
31          .rdy(rdy),
32          .data(data),
33          .insn_ce(insn_ce),
34          .i_ad(i_ad),
35          .int_en(int_en),
36          .d_ad(d_ad),
37          .sw(sw), .sb(sb),
38          .do(do),
39          .lw(lw), .lb(lb)
40      );
```

Figure 32- gr0041.v (1)

```
42      reg isr_active; // 1->(busy in handler)
43      wire is_reti = (insn[15:12] == 4'b1010); //opcode 0xA
44
45      //non-nested interrupts
46      always @(posedge clk) begin
47          if (rst)
48              isr_active <= 0;
49          else if (int)
50              isr_active <= 1; //block new interrupts
51          else if (is_reti && insn_ce)
52              isr_active <= 0; //allow new interrupts after RETI
53      end
54
55
56      reg int_req_last, int_pend;
57
58      //interrupt edge detection
59      always @(posedge clk) begin
60          if (rst) int_req_last <= 0;
61          else int_req_last <= int_req;
62      end
63
64      always @(posedge clk) begin
65          if (rst) int_pend <= 0;
66          else if (int) int_pend <= 0;
67          else if (int_req && ~int_req_last) int_pend <= 1;
68      end
69
70      //interrupt trigger
71      wire int_nxt = int_pend & int_en & ~int & ~isr_active & ~is_reti;
72
73      always @(posedge clk) begin
74          if (rst) int <= 0;
75          else if (insn_ce) int <= int_nxt;
76      end
77
78      assign zero_insn = int_nxt;
79
80  endmodule
```

Figure 33- gr0041.v (2)

3.3 Memory

To support byte-addressable access to the system's memory, two Block Memory Generator from Vivado's IP Catalog were configured as True Dual Port RAM, each one with a depth of 512 address and 8-bits wide (r*amh* and *raml*). The files *ramh.coe* and *raml.coe* are used to fill the respective created memory locations.

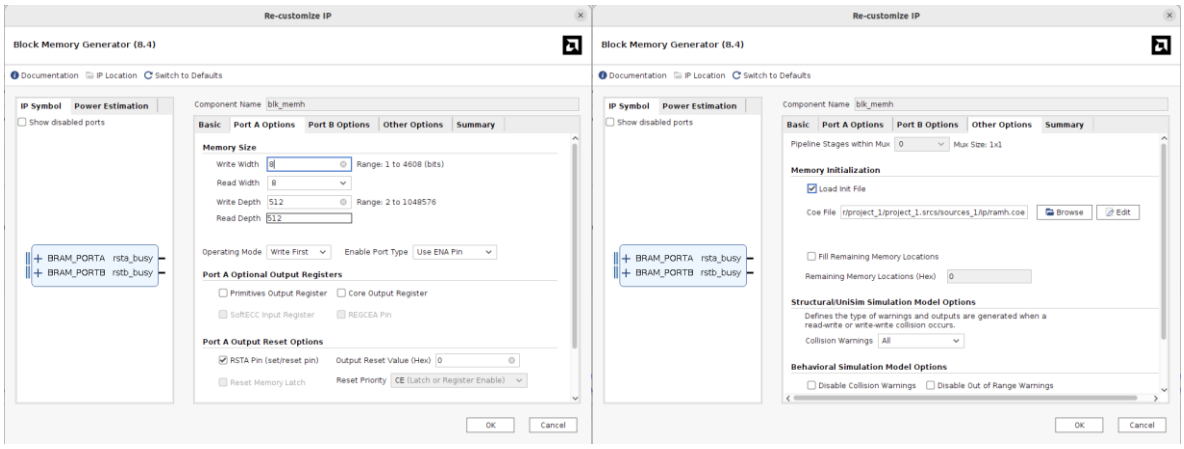

Figure 34- Memory IP configuration

3.4 Peripheral Bus

3.4.1 Bus encoder

The *ctrl_enc* module simplifies peripheral connection by packing all timing and control signals into a single *ctrl* vector, while simultaneously decoding the address to activate the specific peripheral via the *sel* line.

```
1   `timescale 1ns / 1ps
2
3   `ifdef IO
4
5   module ctrl_enc(
6     clk, rst, io, io_ad, lw, lb, sw, sb, ctrl, sel);
7
8     input  clk, rst, io;
9     input  [`AN:0] io_ad;
10    input  lw, lb, sw, sb;
11    output [`CN:0] ctrl;
12    output [`SELN:0] sel;
13
14    // on-chip bus abstract control bus
15    wire [3:0] oe, we;
16
17    //generate byte enables
18    assign oe[0] = io & (lw | lb);
19    assign oe[1] = io & lw;
20    assign oe[2] = 0;
21    assign oe[3] = 0;
22    assign we[0] = io & (sw | sb);
23    assign we[1] = io & sw;
24    assign we[2] = 0;
25    assign we[3] = 0;
26
27    //pack control bus
28    assign ctrl={oe,we,io_ad[`CRAN:0],rst,clk };
29
30    //address decoding
31    assign sel[0] = io & (io_ad[11:8] == 0);
32    assign sel[1] = io & (io_ad[11:8] == 1);
33    assign sel[2] = io & (io_ad[11:8] == 2);
34    assign sel[3] = io & (io_ad[11:8] == 3);
35    assign sel[4] = io & (io_ad[11:8] == 4);
36    assign sel[5] = io & (io_ad[11:8] == 5);
37    assign sel[6] = io & (io_ad[11:8] == 6);
38    assign sel[7] = io & (io_ad[11:8] == 7);
39  endmodule
40  `endif
```

Figure 35- ctrl_enc.v

3.4.2 Bus decoder

The *ctrl_dec* module is instantiated within every peripheral core to ensure protocol compliance. It functions as the counterpart to the encoder, unpacking the *ctrl* vector to recover the local clock, reset, and register address.

```
1  `timescale 1ns / 1ps
2
3  `ifdef IO
4  module ctrl_dec(
5    ctrl, sel, clk, rst, oe, we, ad);
6
7    input  [`CN:0] ctrl;  //abstract control bus
8    input  sel;           //peripheral select
9    output clk;           //clock
10   output rst;           //reset
11   output [3:0] oe;      //byte output enables
12   output [3:0] we;      //byte wire enables
13   output [`CRAN:0] ad;  //ctrl reg addr
14
15   wire [3:0] oe_, we_;
16
17   //unpack control bus
18   assign { oe_, we_, ad, rst, clk } = ctrl;
19
20   //generate output enables
21   assign oe[0] = sel & oe_[0];
22   assign oe[1] = sel & oe_[1];
23   assign oe[2] = sel & oe_[2];
24   assign oe[3] = sel & oe_[3];
25
26   //generate write enables
27   assign we[0] = sel & we_[0];
28   assign we[1] = sel & we_[1];
29   assign we[2] = sel & we_[2];
30   assign we[3] = sel & we_[3];
31 endmodule
32
33 `endif
```

Figure 36- ctrl_dec.v

3.5 16-bit Counter/Timer

Like every peripheral, the Timer instantiates the bus decoder module, as illustrated below.

```
1  `timescale 1ns / 1ps
2  `ifdef IO
3
4  module timer(
5      input[`CN:0] ctrl,
6      inout[`DN:0] data,
7      input  sel, i,
8      input  [15:0] cnt_init,
9      output rdy, irq
10 );
11
12     wire clk, rst;
13     wire [3:0] oe, we;
14     wire [`CRAN:0] ad;
15
16     //bus decoding
17     ctrl_dec d(.ctrl(ctrl), .sel(sel),
18         .clk(clk), .rst(rst), .oe(oe),
19         .we(we), .ad(ad));
20     assign rdy = sel;
```

Figure 37- timer.v (1)

It was implemented a counter that increments upon tick events, derived from either the system clock or external input edges, and automatically reloads from *cnt_init* upon overflow. This overflow event sets the *irq_reg,* asserting the *irq* output signal until it is acknowledged and cleared by software.

```
22      //CR#0: control register 0x8000
23      reg timer, int_en;
24      always @(posedge clk)
25          if (rst) {timer,int_en} <= {2'b11};
26          else if (we[0] & ~ad[1]) {timer,int_en} <= data[1:0];
27
28      //tick Logic
29      reg i_last;
30      always @(posedge clk) i_last <= i;
31      wire tick = (timer&i | ~timer&i&~i_last);
32
33      //counter Logic
34      reg [15:0] cnt;
35      wire [15:0] cnt_nxt;
36      wire v;
37      assign {v,cnt_nxt} = cnt + 1;
38      always @(posedge clk) begin
39          if (rst) cnt <= cnt_init;
40          else if (tick) begin
41              if (v) cnt <= cnt_init;
42              else cnt <= cnt_nxt;
43          end
44      end
45
46      //CR#1: interrupt request register 0x8002
47      reg irq_reg;
48
49      always @(posedge clk)
50          if (rst)
51              irq_reg <= 0;
52          else if (we[0] && ad[1]) //write to CR#1 clears interrupt request
53              irq_reg <= 0;
54          else if (tick && v && int_en) //overflow sets interrupt request
55              irq_reg <= 1;
56
57      assign irq = irq_reg;
58
59      //read Logic
60      assign data[1:0] = oe[0] ? (ad[1]==0 ? {timer,int_en} : irq_reg) : 2'bz;
61
62  endmodule
63  `endif
```

Figure 38- timer.v (2)

3.6 8-bit Parallel I/O Port

The signal *rdy* is driven with *sel*. If the peripheral is selected, it is ready, there are no wait states. On a read operation, the input *i[7:0]* is driven onto *data[7:0].* On a write operation, *data[7:0]* is latched to output o[7:0].

```
`timescale 1ns / 1ps

`ifdef IO

//8-bit parallel i/o peripheral 0x8100
module pario(
    input  [`CN:0] ctrl,
    inout  [`DN:0] data,
    input  sel,
    output rdy,
    input  [7:0] i,
    output [7:0] o
);
    reg [7:0] o;

    wire clk;
    wire [3:0] oe, we;

    //bus decoding
    ctrl_dec d(
        .ctrl(ctrl), .sel(sel),
        .clk(clk), .oe(oe), .we(we)
    );

    assign rdy = sel;

    //write logic
    always @(posedge clk)
        if (we[0])
            o <= data[7:0];

    //read logic
    assign data[7:0] = oe[0] ? i[7:0] : 8'bz;

endmodule

`endif
```

Figure 39- pario.v

3.7 Interrupt Controller

The Interrupt Controller module captures the previous state of the input vector in *irq_prev* and performs a bitwise operation *(irq_in & ~irq_prev)* to identify rising edges *(irq_edge)*.

```
`include "defines.vh"
`timescale 1ns / 1ps

`ifdef IO

module pic(
    input[`CN:0] ctrl,
    inout[`DN:0] data,
    input sel,
    output rdy,
    output int_req,
    input[7:0] irq_in,    //bit 0: timer, bit 1: pario, bit 2: uart
    output reg[15:0] vector_out
);
    wire clk, rst;
    wire[3:0] oe, we;
    wire[`CRAN:0] ad;

    ctrl_dec d(.ctrl(ctrl), .sel(sel),
               .clk(clk), .rst(rst), .oe(oe),
               .we(we), .ad(ad));

    assign rdy = sel;

    reg [7:0] irq_pend; //request pending
    reg [7:0] ie_reg;   //interrupt enable
    reg [7:0] prio_reg; //priority
    reg [7:0] irq_prev; //edge detection

    always @(posedge clk) begin
        if (rst) irq_prev <= 0;
        else irq_prev <= irq_in;
    end

    //posedge detection
    wire [7:0] irq_edge = irq_in & ~irq_prev;

```

Figure 40- pic.v (1)

It implements a "write-1-to-clear" mechanism, allowing software to acknowledge specific interrupts atomically without affecting others.

```
    always @(posedge clk) begin
        if (rst) begin
            irq_pend <= 0;
            ie_reg   <= 0;
            prio_reg <= 0;
        end else begin
            //capture new interrupts, clear handled ones
            if (we[0] && ad[2:0] == 3'b000) begin
                //write 1 to clear specific bit
                irq_pend <= (irq_pend | irq_edge) & ~data[7:0];
            end else begin
                //latch new interrupts
                irq_pend <= irq_pend | irq_edge;
            end

            //write to enable/priority registers
            if (we[0]) begin
                if (ad[2:0] == 3'b010) ie_reg   <= data[7:0]; // 0x8202
                if (ad[2:0] == 3'b110) prio_reg <= data[7:0]; // 0x8206
            end
        end
    end


    wire [7:0] irq_active = irq_pend & ie_reg; //mask pending interrupts with enable register
    assign int_req = |irq_active;              //request to CPU

    //2 bit priority levels (0-3)
    wire [1:0] p_timer = prio_reg[1:0];
    wire [1:0] p_gpio  = prio_reg[3:2];
    wire [1:0] p_uart  = prio_reg[5:4];

    reg [1:0] current_max_prio;

```

Figure 41- pic.v (2)

The following segment implements the arbitration logic responsible for determining the final interrupt vector. The active requests are evaluated in a fixed sequence, updating the *vector_out* only if a subsequent candidate has a higher priority than the current winner. This mechanism ensures a default tie-breaking rule while respecting user configurations.

```
72      always @(*) begin
73          vector_out = 16'h0000;
74          current_max_prio = 0;
75          //if priority register is not configured by user, timer > pario > uart by default
76          //candidate timer 0x0C
77          //sets the "baseline" if active
78          if (irq_active[0]) begin
79              vector_out = 16'h000C;
80              current_max_prio = p_timer;
81          end
82          //candidate pario 0x10
83          //overrides vector_out if no vector set yet, or if priority > timer set by user
84          if (irq_active[1]) begin
85              if ((vector_out == 16'h0000) || (p_gpio > current_max_prio)) begin
86                  vector_out = 16'h0010;
87                  current_max_prio = p_gpio;
88              end
89          end
90          //candidate uart 0x14
91          //overrides vector_out if no vector set yet, or if priority > current winner set by user
92          if (irq_active[2]) begin
93               if ((vector_out == 16'h0000) || (p_uart > current_max_prio)) begin
94                  vector_out = 16'h0014;
95                  current_max_prio = p_uart;
96              end
97          end
98      end
99
100     assign data[7:0] = oe[0] ? (
101         (ad[2:0] == 3'b000) ? irq_pend        :   //status 0x8200
102         (ad[2:0] == 3'b010) ? ie_reg          :   //enable 0x8202
103         (ad[2:0] == 3'b100) ? vector_out[7:0] :   //vector low 0x8204
104         (ad[2:0] == 3'b110) ? prio_reg        :   //priority 0x8206
105         8'b0
106     ) : 8'bz;
107
108     assign data[15:8] = oe[1] ? (
109         (ad[1:0] == 2'b10) ? vector_out[15:8] : //vector high
110         8'b0
111     ) : 8'bz;
112 endmodule
113 `endif
```

Figure 42- pic.v (3)

3.8 UART

The UartController top module interconnects the BaudRateGen, UartTransmitter, and UartReceiver sub-modules.

```
`include "defines.vh"
`timescale 1ns / 1ps
`ifdef IO

module UartController
(
    input [`CN:0] ctrl,
    inout [`DN:0] data,
    input sel,
    output rdy,
    input rx_pin,
    output tx_pin,
    output irq_out
);

    wire clk, rst;
    wire [3:0] oe, we;
    wire [`CRAN:0] ad;

    ctrl_dec d(
        .ctrl(ctrl), .sel(sel),.clk(clk), .rst(rst),
        .oe(oe), .we(we), .ad(ad));

    assign rdy = sel;
    wire internal_tick, tx_complete, rx_complete;
    wire [7:0] rx_byte_wire;
    reg [15:0] reg_baudrate;    //0x8302
    reg [7:0]  reg_tx_buffer;   //0x8304
    reg ctrl_en_module;         //0x8300 bit 0
    reg ctrl_en_rx;             //0x8300 bit 1
    reg ctrl_start_tx;          //0x8300 bit 2
    reg last_tx_done;

    BaudRateGen baud_gen_inst (.rst(rst),.clk(clk),.en(ctrl_en_module),
        .divisor(reg_baudrate),.baud_tick(internal_tick));

    UartTransmitter tx_inst (.baud_tick(internal_tick),.start_tx(ctrl_start_tx),
        .data_byte(reg_tx_buffer),.rst(rst),.tx_serial(tx_pin),.tx_ready(tx_complete));

    UartReceiver rx_inst (.baud_tick(internal_tick),.rst(rst),.rx_enable(ctrl_en_rx),
        .rx_serial(rx_pin),.byte_out(rx_byte_wire),.rx_flag(rx_complete));

```

Figure 43- UartController.v (1)

```
44      //write (CPU -> UART)
45      always @(posedge clk) begin
46          if (rst) begin
47              ctrl_en_module <= 0;
48              ctrl_start_tx  <= 0;
49              ctrl_en_rx     <= 0;
50              reg_tx_buffer  <= 0;
51              reg_baudrate   <= 0;
52              last_tx_done   <= 0;
53          end
54          else begin
55              //clear tx start bit when transmission finishes
56              if ((tx_complete == 1'b1) && (last_tx_done == 1'b0)) begin
57                  ctrl_start_tx <= 0;
58                  last_tx_done  <= tx_complete;
59              end
60              else begin
61                  last_tx_done <= tx_complete;
62              end
63
64              //bus write operation
65              if (sel && (we[0] || we[1])) begin
66                  case (ad[2:1])
67                      2'b00: begin //control register 0x8300
68                          {ctrl_start_tx, ctrl_en_rx, ctrl_en_module} <= data[2:0];
69                      end
70
71                      2'b01: begin //baudrate divisor 0x8302
72                          reg_baudrate <= data;
73                      end
74
75                      2'b10: begin //tx data 0x8304
76                          reg_tx_buffer <= data[7:0];
77                      end
78                  endcase
79              end
80          end
81      end
```

Figure 44- UartController.v (2)

```
83      //read logic (UART -> CPU)
84      reg [15:0] read_val;
85
86      always @(*) begin
87          case (ad[2:1])
88              2'b00: begin //status register 0x8300
89                  read_val = {11'b0, rx_complete, tx_complete, ctrl_start_tx, ctrl_en_rx, ctrl_en_module};
90              end
91
92              2'b01: begin //read baudate divisor 0x8302
93                  read_val = reg_baudrate;
94              end
95
96              2'b10: begin //rx data read 0x8304
97                  read_val = {8'b0, rx_byte_wire};
98              end
99
100             default: read_val = 16'h0000;
101         endcase
102     end
103
104     assign data = (sel && (oe[0] || oe[1])) ? read_val : 16'bz;
105
106     //interrupt on valid rx data
107     assign irq_out = rx_complete;
108
109 endmodule
110
111 `endif
```

Figure 45- UartController.v (3)

3.8.1 Baudrate Generator

The BaudRateGen module utilizes a 16-bit down-counter driven by a gated clock signal *(clk & en)*. Upon reaching the count of 1, *baud_tick* output is toggled and the internal register is reloaded with half the input divisor value *(divisor[15:1])*. This technique generates a square wave clock signal with a total period exactly matching the specified divisor cycles, which is then used to synchronize the bit-shifting operations in the Transmitter and Receiver modules.

```
1   `timescale 1ns / 1ps
2
3   module BaudRateGen
4   (
5       input rst,
6       input clk,
7       input en,
8       input [15:0] divisor, //divisor value
9       output reg baud_tick
10  );
11      //gated clock for enable control
12      wire gated_clk;
13      assign gated_clk = clk & en;
14
15      reg [15:0] cnt_reg;
16
17      always @(posedge gated_clk) begin
18          if(rst) begin
19              baud_tick <= 0;
20              cnt_reg   <= 1;
21          end
22          else if (cnt_reg == 1) begin
23              //toggle output and reload
24              baud_tick <= ~baud_tick;
25              cnt_reg   <= {1'b0, divisor[15:1]};
26          end
27          else begin
28              cnt_reg <= cnt_reg - 1;
29          end
30      end
31  endmodule
```

Figure 46- BaudRateGen.v

3.8.2 Receiver

This module implements the FSM previously designed for data reception.

```
1  `timescale 1ns / 1ps
2  module UartReceiver(input baud_tick, input rst, input rx_enable, input rx_serial,
3      output reg [7:0] byte_out,
4      output reg rx_flag);
5
6      parameter STATE_WAIT = 2'b00, STATE_READ = 2'b01, STATE_END  = 2'b10;
7      reg [3:0] sample_cnt;
8      reg [7:0] rx_buffer;
9      reg [1:0] rx_state;
10
11     always@(posedge baud_tick or posedge rst) begin
12         if(rst) begin
13             rx_state <= STATE_WAIT;
14             rx_flag  <= 0;
15             byte_out <= 0;
16         end
17         else if(rx_enable == 1'b1) begin
18             case(rx_state)
19                STATE_WAIT: begin
20                     rx_flag <= 0;
21                     //detect start bit (falling edge)
22                     if(rx_serial == 1'b0) begin
23                         sample_cnt <= 0;
24                         rx_buffer  <= 0;
25                         rx_state   <= STATE_READ;
26                     end end
27                STATE_READ: begin //shift in data
28                     rx_buffer  <= {rx_serial, rx_buffer[7:1]};
29                     sample_cnt =  sample_cnt + 1;
30                     if(sample_cnt[3]) //8 bits received
31                         rx_state <= STATE_END;
32                end
33                STATE_END: begin //output valid data
34                     byte_out <= rx_buffer;
35                     rx_flag  <= 1;
36                     rx_state <= STATE_WAIT;
37                end
38                default:
39                     rx_state <= STATE_WAIT;
40             endcase end
41         end
42 endmodule
```

Figure 47- UartReceiver.v

3.8.3 Transmitter

This module implements the FSM previously designed for data transmission.

```
1   `timescale 1ns / 1ps
2
3   module UartTransmitter(input baud_tick,input start_tx, rst,
4       input [7:0] data_byte,
5       output reg tx_serial, output reg tx_ready);
6
7       parameter STATE_IDLE=2'b00, STATE_START=2'b01, STATE_SHIFT=2'b10, STATE_STOP=2'b11;
8       reg [1:0] current_state;
9       reg [7:0] shift_reg;
10      reg [2:0] bit_pos;
11
12      always@(posedge baud_tick or posedge rst) begin
13          if(rst) begin
14              current_state = STATE_IDLE;
15              tx_serial <= 1'b1;
16              tx_ready  <= 0; end
17          else begin
18              case(current_state)
19                  STATE_IDLE: begin   //wait for start command
20                      if((start_tx == 1'b1)) begin
21                         tx_ready <= 0;
22                         current_state = STATE_START; end end
23                  STATE_START: begin
24                      tx_serial <= 0;          //send start bit (0)
25                      shift_reg <= data_byte;  //load buffer
26                      bit_pos   <= 7;
27                      current_state = STATE_SHIFT; end
28                  STATE_SHIFT: begin
29                      tx_serial <= shift_reg[0]; //send LSB
30                      bit_pos   <= bit_pos - 1;
31                      shift_reg = {1'b0, shift_reg[7:1]}; //shift right
32
33                      if(bit_pos == 0)
34                          current_state = STATE_STOP; end
35                 STATE_STOP: begin
36                     tx_ready  <= 1;
37                     tx_serial <= 1;     //send stop bit (1)
38                     current_state = STATE_IDLE; end
39                 default:
40                      current_state = STATE_IDLE;
41              endcase end end
42  endmodule
```

Figure 48- UartTransmitter.v

3.9 Top-level module

The *soc* module interconnects all the implemented system components. The code excerpt below illustrates the instantiation of the GR0041 processor core. The *loaded* register inserts necessary wait states during synchronous RAM reads, while simultaneously generating an *io* flag to distinguish peripheral accesses from standard memory operations based on the address bus MSB.

```
`include "defines.vh"

module soc(
    input  clk,rst,
    input[7:0] par_i,
    output[7:0] par_o,
    input  uart_rx,
    output uart_tx
);
    wire[`AN:0] i_ad, d_ad;
    wire[`N:0] insn, do, data;
    wire zero_insn, rdy, sw, sb, lw, lb, cpu_req, timer_irq, uart_irq;
    wire [15:0] pic_vector;
    wire gpio_irq=par_i[0]; //mapped to board physical button
    wire[7:0] irq_sources = {5'b0, uart_irq, gpio_irq, timer_irq};

    gr0041 p(
        .clk(clk), .rst(rst),.i_ad_rst(16'h0020),
        .int_req(cpu_req),.insn_ce(insn_ce),
        .i_ad(i_ad),.insn(insn), .hit(~rst),
        .zero_insn(zero_insn),.d_ad(d_ad),
        .rdy(rdy),.sw(sw), .sb(sb),.do(do),
        .lw(lw), .lb(lb), .data(data),.vector_in(pic_vector));

    reg loaded;
    always @(posedge clk)
        if (rst) loaded <= 0;
        else if (insn_ce) loaded <= 0;
        else loaded <= (lw|lb);
`ifdef IO
    wire io_nxt = d_ad[`AN];
`else
    wire io_nxt = 0;
`endif
    reg io; //peripheral I/O valid access
    always @(posedge clk)
        if (rst)
            io <= 0;
        else if (insn_ce)
            io <= 0;
        else
            io <= io_nxt;
```

Figure 49- soc.v (1)

The bidirectional data bus is governed by a multiplexer that routes read data from the memory or drives the processor's write data *(do)* onto the bus during peripheral I/O, ensuring correct byte alignment for both word and byte operations.

```
43      wire io_rdy;
44      assign rdy = ~io_nxt & ~((lw|lb) & ~loaded) | io & io_rdy | rst;
45
46      wire h_we = ~rst & (sw | sb & ~d_ad[0]);    //byte addressable memory acess
47      wire l_we = ~rst & (sw | sb &  d_ad[0]);
48      wire [7:0] do_h = sw ? do[15:8] : do[7:0];
49      wire [`N:0] di;
50
51      blk_memh ramh(
52          .rsta(zero_insn), .wea(1'b0),.ena(insn_ce), .clka(clk),
53          .addra(i_ad[9:1]), .dina(8'b0),.douta(insn[15:8]),.rstb(rst),
54          .web(h_we),.enb(1'b1), .clkb(clk),.addrb(d_ad[9:1]), .dinb(do_h),
55          .doutb(di[15:8]));
56
57      blk_meml raml(
58          .rsta(zero_insn), .wea(1'b0),.ena(insn_ce), .clka(clk),
59          .addra(i_ad[9:1]), .dina(8'b0),.douta(insn[7:0]),
60          .rstb(rst), .web(l_we),.enb(1'b1), .clkb(clk),
61          .addrb(d_ad[9:1]), .dinb(do[7:0]),.doutb(di[7:0]));
62
63      wire w_oe = ~io & lw;
64      wire l_oe = ~io & (lb & d_ad[0] | lw);
65      wire h_oe = ~io & (lb & ~d_ad[0]);
66      wire io_wr_word = io & sw;
67      wire io_wr_byte = io & sb;
68
69      //bus data 8MSB
70      assign data[15:8] = (w_oe)        ? di[15:8] :
71                          (lb & ~io)    ? 8'b0     :
72                          (io_wr_word)  ? do[15:8] :
73                          (io_wr_byte)  ? do[7:0]  :
74                          8'bz;
75
76      //bus data 8LSB
77      assign data[7:0]  = (l_oe)        ? di[7:0]  :
78                          (h_oe)        ? di[15:8] :
79                          (io_wr_word | io_wr_byte) ? do[7:0] :
80                          8'bz;
```

Figure 50- soc.v (2)

The *soc* module instantiates all the system peripherals.

```
82      //io controller
83  `ifdef IO
84      reg [`AN:0] io_ad;
85      always @(posedge clk) io_ad <= d_ad;
86      wire [`CN:0] ctrl;
87      wire [`SELN:0] sel;
88
89      ctrl_enc enc(
90          .clk(clk), .rst(rst), .io(io),.io_ad(io_ad),.lw(lw),
91          .lb(lb), .sw(sw),.sb(sb), .ctrl(ctrl), .sel(sel));
92
93      wire [`SELN:0] per_rdy;
94      assign io_rdy = | (sel & per_rdy);
95
96      timer timer(
97          .ctrl(ctrl),.data(data),.sel(sel[0]), .rdy(per_rdy[0]),
98          .irq(timer_irq), .i(1'b1),.cnt_init(16'h0001));
99
100     pario par(
101         .ctrl(ctrl), .data(data),.sel(sel[1]),
102         .rdy(per_rdy[1]),.i(par_i), .o(par_o));
103
104     pic interrupt_controller(
105         .ctrl(ctrl), .data(data),.sel(sel[2]), .rdy(per_rdy[2]),
106         .int_req(cpu_req),.irq_in(irq_sources),.vector_out(pic_vector));
107
108     UartController uart (
109         .ctrl(ctrl),.data(data),.sel(sel[3]),
110         .rdy(per_rdy[3]),.rx_pin(uart_rx),
111         .tx_pin(uart_tx), .irq_out(uart_irq)
112     );
113 `else
114     assign cpu_req = 0;
115     assign io_rdy = 0;
116 `endif
117 endmodule
```

Figure 51- soc.v (3)

3.10 Python Assembler

A custom Assembler was developed using Python. This tool automates the translation of assembly instructions into 16-bit machine code according to the defined ISA. The script parses the assembly source file and splits the instructions into high and low bytes, formatting them as hexadecimal values to populate the *ramh.coe* and *raml.coe* files directly.

This approach significantly reduced the testing time. In the following figures, the developed Python algorithm is shown.

```python
assembler.py ×   codigo.asm
assembler.py > assemble_line
import sys
import re
import os

#-----------------------------------------------------------------------------
#   instruction set definition
#-----------------------------------------------------------------------------
OPCODES = {
    'jal': 0x0, 'addi': 0x1,
    'add': 0x2, 'sub': 0x2, 'and': 0x2, 'xor': 0x2, 'adc': 0x2, 'sbc': 0x2, 'cmp': 0x2, 'srl': 0x2, 'sra': 0x2,
    'rsubi': 0x3, 'adci': 0x3, 'rsbci': 0x3, 'andi': 0x3, 'xori': 0x3, 'rcmpi': 0x3,
    'lw': 0x4, 'lb': 0x5, 'sw': 0x6, 'sb': 0x7,
    'imm': 0x8,
    'br': 0x9, 'beq': 0x9, 'bne': 0x9, 'bc': 0x9, 'bnc': 0x9, 'bv': 0x9, 'bnv': 0x9,
    'blt': 0x9, 'bge': 0x9, 'ble': 0x9, 'bgt': 0x9, 'bltu': 0x9, 'bgeu': 0x9, 'bleu': 0x9, 'bgtu': 0x9,
    'reti': 0xA
}

ALU_FN = {
    'add': 0x0, 'sub': 0x1, 'and': 0x2, 'xor': 0x3, 'adc': 0x4, 'sbc': 0x5, 'cmp': 0x6, 'srl': 0x7, 'sra': 0x8,
    'rsubi': 0x1, 'adci': 0x4, 'rsbci': 0x5, 'andi': 0x2, 'xori': 0x3, 'rcmpi': 0x6
}

BRANCH_COND = {
    'br': 0x0, 'beq': 0x2, 'bne': 0x3, 'bc': 0x4, 'bnc': 0x5, 'bv': 0x6, 'bnv': 0x7,
    'blt': 0x8, 'bge': 0x9, 'ble': 0xA, 'bgt': 0xB, 'bltu': 0xC, 'bgeu': 0xD, 'bleu': 0xE, 'bgtu': 0xF
}
```

Figure 52- assembler.py (1)

```python
#-----------------------------------------------------------------------------
#   parser
#-----------------------------------------------------------------------------
def parse_reg(s): return int(s.lower().strip()[1:])
def parse_imm(s):
    s = s.strip()
    if s.lower().startswith('0x') or "16'h" in s: return int(s.replace("16'h", "0x"), 16)
    elif s.lower().startswith('0b'): return int(s, 2)
    return int(s)

def assemble_line(line, current_index, labels):
    parts = re.split(r'\s+', line, 1)
    mnem = parts[0].lower()
    args = [x.strip() for x in re.split(r',|\(|\)', parts[1])] if len(parts) > 1 else []

    op = OPCODES[mnem]

    if mnem == 'jal': return (op<<12) | (parse_reg(args[0])<<8) | (parse_reg(args[2])<<4) | (parse_imm(args[1])&0xF)
    if mnem == 'addi': return (op<<12) | (parse_reg(args[0])<<8) | (parse_reg(args[1])<<4) | (parse_imm(args[2])&0xF)
    if mnem in ['lw', 'sw', 'lb', 'sb']: return (op<<12) | (parse_reg(args[0])<<8) | (parse_reg(args[2])<<4) | (parse_imm(args[1])&0xF)
    if mnem in ALU_FN and op==2: return (op<<12) | (parse_reg(args[0])<<8) | (parse_reg(args[1])<<4) | (ALU_FN[mnem]&0xF)
    if mnem in ALU_FN and op==3: return (op<<12) | (parse_reg(args[0])<<8) | (ALU_FN[mnem]<<4) | (parse_imm(args[1])&0xF)
    if mnem == 'imm': return (op<<12) | (parse_imm(args[0])&0xFFF)

    if mnem == 'reti':
        reg_val = 0
        if len(args) > 0:
            reg_val = parse_reg(args[0])
            if reg_val != 0:
                print(f"Line {current_index*2:04X}]: 'reti' must use r0")
        return (op << 12) | (reg_val << 8) #returns 0xA000

    if mnem in BRANCH_COND:
        tgt = args[0]
        if tgt in labels:
            disp = labels[tgt] - current_index
        else:
            disp = parse_imm(tgt)
        return (op<<12) | (BRANCH_COND[mnem]<<8) | (disp&0xFF)
    return 0
```

Figure 53- assembler.py (2)

```python
def main():

    input_file = '/home/tiago/Desktop/assembler/codigo.asm'
    output_path = '/home/tiago/Desktop/processador/project_1/project_1.srcs/sources_1/ip/'

    try:
        with open(input_file, 'r') as f: lines = f.readlines()
    except:
        print("Error: codigo.asm not found")
        return

    labels = {}
    word_index = 0
    instruction_map = {}

    # 1-symbol table
    for line in lines:
        line = line.split('//')[0].strip()
        if not line: continue
        if line.startswith('.org'):
            addr_val = parse_imm(line.split()[1])
            word_index = addr_val // 2
            print(f"--> .org 0x{addr_val:X} (CPU) -> RAM Index {word_index}")
            continue
        if line.endswith(':'):
            labels[line[:-1]] = word_index
        else:
            instruction_map[word_index] = line
            word_index += 1

    max_index = word_index
    coe_data = []

```

Figure 54- assembler.py (3)

```python

    # 2-assembly
    for i in range(max_index):
        if i in instruction_map:
            try:
                hex_val = assemble_line(instruction_map[i], i, labels)
                coe_data.append(hex_val)
                print(f"[PC {i*2:04X}] {instruction_map[i]:<25} -> {hex_val:04X}")
            except Exception as e:
                print(f"Error line '{instruction_map[i]}': {e}")
                return
        else:
            coe_data.append(0x1000)

    if not os.path.exists(output_path): output_path = "."

    with open(os.path.join(output_path, 'ramh.coe'), 'w') as fh, \
         open(os.path.join(output_path, 'raml.coe'), 'w') as fl:
        header = "memory_initialization_radix=16;\nmemory_initialization_vector=\n"
        fh.write(header); fl.write(header)
        for i, word in enumerate(coe_data):
            sep = ",\n" if i < len(coe_data)-1 else ";\n"
            fh.write(f"{(word>>8)&0xFF:02X}{sep}")
            fl.write(f"{word&0xFF:02X}{sep}")

    print("\nramh.coe and raml.coe generated")


if __name__ == "__main__":
    main()

```

Figure 55- assembler.py (4)

4 Verification

To test all the system's functionalities, a test was performed using the Zybo Z7-10 board to:

- Toggle an LED to signalize Timer interrupts
- Toggle another LED in response to push-button presses (GPIO)
- Echo keyboard inputs back to the host terminal using GTKTerm (UART)

4.1 Test Code

The following Assembly code was developed. In the main routine, UART is configured to a baudrate of *9600bps,* and the peripheral interrupts are enabled.

codigo.asm

```
.org 0x000C
timer_vector:
    br timer_handler

.org 0x0010
gpio_vector:
    br pario_handler

.org 0x0014
uart_vector:
    br uart_handler

.org 0x0020
main:

    //setup baudrate divisor (0x1458)
    imm 0x145
    addi r2, r1, 8
    imm 0x830
    addi r3, r1, 2
    sw r2, 0(r3)    // baudrate register 0x8302 = 0x1458

    //setup uart control (rx=1, en=1) -> 3
    addi r2, r1, 3
    imm 0x830
    addi r3, r1, 0
    sw r2, 0(r3)    //control register 0x8300 = 3

    // timer+pario+uart = 4+2+1 = 7
    imm 0x820
    addi r2, r1, 2
    addi r3, r1, 7
    sw r3, 0(r2)    //interrupt enable register 0x8202 = 7

loop:
    br loop
```

Figure 56- Assembly test code (1)

The UART handler implements an echo function, and the GPIO handler toggles an LED whenever a button press triggers the interrupt.

```
uart_handler:
    xor r1, r1
    imm 0x830
    addi r1, r1, 0  // r1 = 0x8300

    //echo: read -> write
    lw r2, 4(r1)
    sw r2, 4(r1)

    //start tx (keep rx enabled) -> 7 (111b)
    xor r3, r3
    addi r3, r3, 7
    sw r3, 0(r1)    //start TX

    //clear pic (uart irq = bit 2 -> 4)
    xor r1, r1
    imm 0x820
    addi r1, r1, 0
    xor r2, r2
    addi r2, r2, 4
    sw r2, 0(r1)

    reti r0

pario_handler:
    xor r1, r1
    imm 0x810
    addi r1, r1, 0  //r1 = 0x8100

    xori r15, 4      //toggle led2
    sw r15, 0(r1)

    //clear pic (gpio irq = bit 1 -> 2)
    xor r1, r1
    imm 0x820
    addi r1, r1, 0
    xor r2, r2
    addi r2, r2, 2
    sw r2, 0(r1)

    reti r0
```

Figure 57- Assembly test code (2)

The timer handler toggles an LED every 500 timer interrupts (to be human-eye visible).

```
timer_handler:
    addi r9, r9, 1 //increment counter

    //(counter == 500) ?
    xor r1, r1
    imm 0x01F
    addi r1, r1, 4
    cmp r9, r1
    bne clear_timerInt //false

    //true
    xor r9, r9
    xor r1, r1
    imm 0x810
    addi r1, r1, 0  // r1 = 0x8100
    xori r15, 1      //toggle led0
    sw r15, 0(r1)

clear_timerInt:
    //clear interrupt request
    xor r1, r1
    imm 0x800
    addi r1, r1, 2
    xor r2, r2
    sw r2, 0(r1)    // Escreve 0 em 0x8002

    //clear pic (timer irq = bit 0 -> 1)
    xor r1, r1
    imm 0x820
    addi r1, r1, 0
    xor r2, r2
    addi r2, r2, 1
    sw r2, 0(r1)

    reti r0
```

Figure 58- Assembly test code (3)

4.2 Vivado Block Design

After injecting this program to the RAM, the System-on-Chip was packaged into an IP and integrated in the following block design. The external pins of this design were mapped to the board's physical pins in the constraints file. The outputs *tx_0* and *rx_0* were connected to an FTDI module.

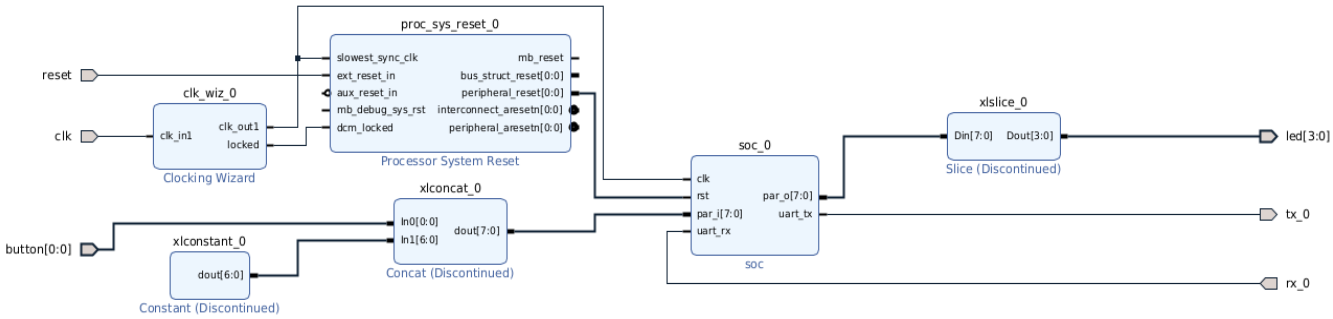

Figure 59- Block Design (Vivado)

4.3 Results

The bitstream was generated, and the system behaved as expected, demonstrating the correct functioning of all the peripherals, including the Interrupt Controller.

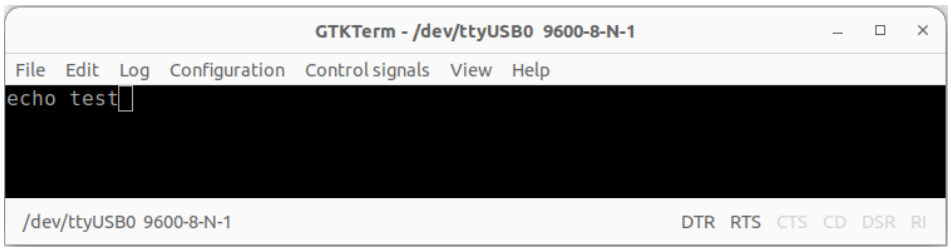

Figure 60- Echo test using GTKTerm

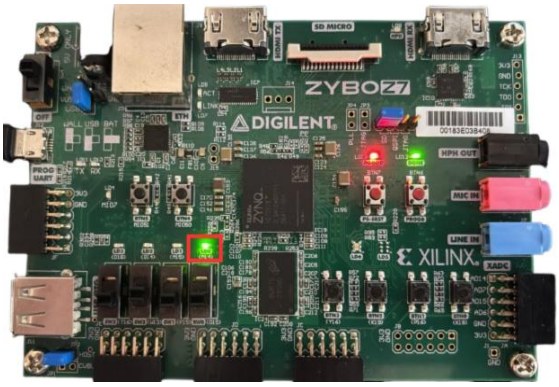

Figure 61- LED toggling every 500 timer interrupts

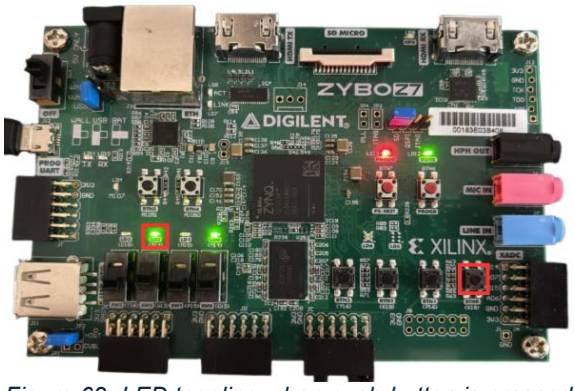

Figure 62- LED toggling when push-button is pressed

5 Conclusion

The development of this System-on-Chip represents the successful integration of the lectured computer architecture theoretical concepts into a functional, synthesized hardware design. This project allowed us to consolidate our Verilog knowledge and proficiency with the Vivado design flow, leaving us well-prepared for future challenges.

The work is set to continue in the second semester, where a pipeline architecture will be implemented to significantly improve the processor's performance.

6 References

[1] http://fpgacpu.org/gr/index.html

[2] https://docs.amd.com/v/u/en-US/ug1793-vivado-design-suite